

What the App Can Do Under the Hood

A plain-language tour of the data engine behind O&M Cane Training — how children's records are stored, protected, exported, and prepared for cloud — written for design & product, with the real code shown where it helps.

Prepared for	App	Stack	Status
Design / Product lead	org.omcane.trainer	Plain HTML/JS + Capacitor 8 (Android)	Local-first, on-device, shipping-ready

This document explains **everything the backend currently does** — not the design or the activity content, but the machinery that holds a child's profile and every recorded session. The short version: the app keeps all data **on the device**, every save is **verified** so nothing silently disappears, and a teacher can pull everything out as a **spreadsheet** at any time. Cloud and video sync are designed but deliberately not switched on yet.

01 The big picture

One small idea organises the whole backend: a single doorway that everything passes through.

There is exactly **one place** in the code that talks to storage. We call it the **Store seam**. Every other part of the app — saving a result, loading a child, deleting a record, building the export — goes *through* it and never touches the device storage directly.

HOW DATA MOVES THROUGH THE APP



WHY A DESIGNER SHOULD CARE

Because storage lives behind one doorway, swapping the engine underneath — from a browser store to native phone storage, and later to the cloud — was a

4-LINE CHANGE

, not a rewrite. New features (sync, backup, multi-device) plug into the same doorway without disturbing the screens you design.

02 Where data actually lives

Two engines, one interface. The app picks the right one automatically at launch.

Engine	Used when	What it is
Capacitor Preferences	On the real Android app	Native phone storage. Survives closing the app, restarting the phone, and updates. This is the real product path.
Browser localStorage	Testing in a browser	A fallback so the app still runs on a laptop during development. Same behaviour, no native phone needed.

The app detects which environment it's in **once**, at startup, and from then on no other code needs to know or care which engine is live.

BOOT — DETECT THE ENGINE, THEN LOAD EVERYTHING INTO MEMORY ONCE

```

async init(){
  // Capacitor injects window.Capacitor on a real device.
  const cap = window.Capacitor;
  if(cap && cap.isNativePlatform() && cap.Plugins.Preferences){
    this._native = true;           // → use native phone storage
    this._prefs = cap.Plugins.Preferences;
  }
  this._cache = await this._backendEntries(); // load all saved data into memory
  this._ready = true;
}

```

03 The clever bit — fast reads, safe writes

Native storage is slow and asynchronous. Screens need data instantly. Here's how both are true at once.

Native phone storage answers **slowly** (every read is a "wait for it" operation). But when a screen draws — a list of children, past results — it needs data **right now**. The fix: at launch the app copies all saved data into a fast **in-memory cache**. Screens read instantly from the cache; the slow part only happens when *writing*.

READS VS. WRITES — TWO DIFFERENT SPEEDS ON PURPOSE

READS (instant)

Screen asks cache → answer returned immediately. No waiting.

WRITES (careful)

1 · update cache → 2 · write to device → 3 · read it back to confirm

The write-verify guarantee

This is the most important safety property in the whole backend. When the app saves, it doesn't just *assume* the write worked — it **reads the value back** and checks it matches. If it doesn't, the save reports failure and the teacher is told. For a tool holding children's disability data, a save that **silently** vanishes is the worst possible outcome, so the app refuses to pretend.

EVERY SAVE: WRITE THROUGH, THEN READ BACK AND VERIFY

```
async setJSON(key, value){
  const serialised = JSON.stringify(value);
  this._cache[key] = serialised;           // 1. update fast cache
  const ok = await this._backendSet(key, serialised); // 2. write to device
  if(!ok) return false;
  const check = await this._backendGet(key);    // 3. read it back
  return check === serialised;                 // true ONLY if it really stored
}
```

WHAT THIS MEANS FOR THE USER EXPERIENCE

When a save fails (e.g. storage full), the teacher sees a clear message — *"Could not save — export your data, then try again"* — instead of losing a session without knowing. The reliability you'd want for medical-style records is built in at the lowest level.

04 What the app stores

Three kinds of data, each in its own clearly-named space behind the seam.

Data	Stored as	Contents
Child profiles	key profiles	Name, date of birth, height, weight, dominant hand, and a small photo. Age is <i>calculated</i> from DOB, never frozen, so it's never stale.
Session records	keys rec_{activity}	One bucket per activity. Each record links to a child, a timestamp, and the result values the teacher logged.
Settings	small keys	The active child, the chosen SOP audio language, and a "welcome seen" flag.

Photos are shrunk before storage

A chosen photo is automatically **downscaled to ~160px and re-compressed** before being saved. This keeps storage small *and* deliberately limits how identifiable a stored image is — a quiet privacy choice baked into the code.

PHOTO DOWNSCALE — PRIVACY AND STORAGE SIZE IN ONE STEP

```
img.onload = () => {
  let { width:w, height:h } = img;
  if(w > h && w > max){ h = Math.round(h*max/w); w = max; } // cap longest side
  const canvas = document.createElement('canvas');
  canvas.width = w; canvas.height = h;
  canvas.getContext('2d').drawImage(img, 0, 0, w, h);
  resolve(canvas.toDataURL('image/jpeg', 0.7)); // small, low-res JPEG
};
```

05 Deleting data — a real feature, not a debug switch

Because this is children's disability data, removing it cleanly is a first-class capability.

A teacher can delete **one wrong record**, remove **a whole child** (which also sweeps away every session linked to them), or **wipe the device** entirely. Every destructive action asks for confirmation and points the teacher at the CSV export first, because the export is the only backup.

Action	What it removes
Delete one result	A single session record; if it was the last one in that activity, the empty bucket is removed too.
Delete a child	The profile <i>and</i> every session record across every activity that references them; clears the active selection if needed.
Clear all data	Every profile and every record on the device. (Keeps only the "welcome seen" flag so the app doesn't reset to the intro screen.)

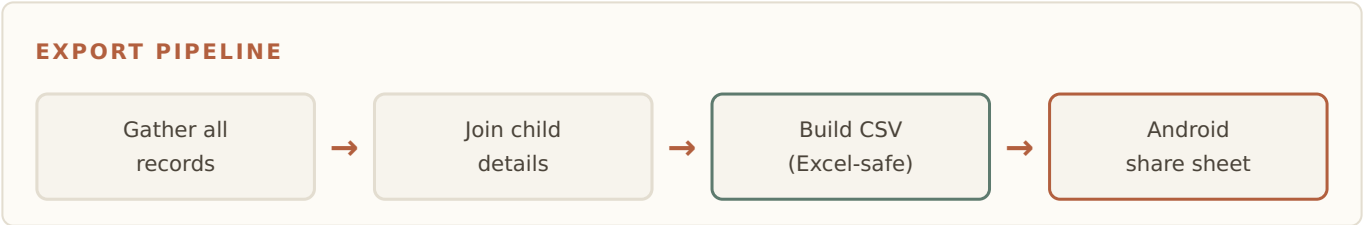
COMPLIANCE ANGLE

"Delete a child and all their data" is exactly the kind of right-to-erasure capability that India's DPDP Act expects for children's data. It already exists, working, at the data layer.

06 Getting data out — CSV export

The escape hatch. Until cloud sync exists, this is how a teacher's data leaves the device.

The app gathers **every record across every activity**, joins each one to its child's details, and builds a single spreadsheet. Teachers open it in Excel or Google Sheets — not a technical backup file, a real working spreadsheet. Two careful touches: a **BOM marker** so Excel reads Indian-language names correctly, and proper **quoting** so commas in notes don't break columns.



On a real phone the app can't use a browser "download." Instead it writes the file and hands it to the **Android share sheet**, so the teacher saves it to Files, Drive, or email — the native equivalent of a download. If that fails, it says so loudly.

NATIVE EXPORT — WRITE THE FILE, THEN OPEN THE OS SHARE SHEET

```
const writeRes = await Filesystem.writeFile({
  path: filename, data: csv, directory: 'CACHE', encoding: 'utf8'
});
const uriRes = await Filesystem.getUri({ path: filename, directory: 'CACHE' });
await Share.share({
  title: 'O&M records export',
  url: uriRes.uri, // the file the teacher saves/sends
  dialogTitle: 'Save or send your records CSV'
});
```

07 The native shell & audio

How the web app becomes a real Android app, and how SOP narration is wired.

NATIVE WRAPPER

The app is wrapped with **Capacitor 8** into a real Android app (ID org.omcane.trainer). Four native plugins are in use:

- **Preferences** — durable storage
- **Filesystem** — writing the export file
- **Share** — the save/send sheet
- **SplashScreen** — the launch screen

SOP AUDIO NARRATION

Each activity can carry narration in **Hindi, Tamil, and Bengali**. The app builds the audio filename from the activity + chosen language, and the choice is remembered through the same Store seam. Missing audio falls back gracefully to a "not generated yet" note rather than breaking.

VOICE-FIRST READINESS

The language-switching and audio plumbing is already in place — the foundation for the audio-first experience the end users (visually impaired children and their teachers) will need. Audio files are generated separately and dropped in; the app just plays them.

08 Capability scorecard

Everything above, summarised — what's live, what's partial, what's next.

Capability	Status	Note
On-device durable storage (native)	Live	Confirmed to survive cold restarts on the emulator.
Write-verify (no silent data loss)	Live	Every save read back and confirmed.
Child profiles + photos	Live	Photos auto-downscaled for privacy & size.
Session records per activity	Live	Linked to child + timestamp.
Full deletion (record / child / all)	Live	Erasability for compliance.
CSV export via Android share sheet	Live	Excel-safe, Indian-language-safe.
Multilingual SOP audio playback	Partial	Plumbing done; audio files generated per activity.
Real Android phone testing (USB)	Next	Verified on emulator; physical device pending.
Cloud storage / video (Supabase)	Designed	Architecture settled; switched on after the data-pooling & consent decision.

One thing to flag for you as manager: the backend is deliberately **local-first**. Cloud sync and session-video storage are fully designed but **not switched on**, because turning them on crosses a compliance threshold (consent + how records are pooled). That's a product/policy decision waiting on you and the content team — not a missing piece of engineering. Everything that can ship safely today already works.

Prepared from the live source (`index.html`, `capacitor.config`, `package.json`). Code shown is lightly trimmed for readability; behaviour is exact. The activity content itself lives in `activities.js` and is owned by the content team — untouched by anything described here.